МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №4.2
по дисциплине
«Программирование»
по теме: "Динамические структуры данных"

Выполнил студент гр. ИВТб-1301-06-00 _____/Габдулбариев Т.Р./
Проверил доцент кафедры ЭВМ _____/Долженкова М.Л./

Киров
2026

Цель

Научиться создавать простые графические интерфейсы с использованием библиотеки tkinter, организовывать обработку событий и интегрировать ранее разработанные программные модули в единое приложение. Закрепить понимание работы выбранной структуры данных через её визуальное представление.

Задание

- На основе реализации структуры данных из лабораторной работы 4.1. необходимо разработать оконное приложение с графическим интерфейсом на Python.
- Все операции со структурой данных должны выполняться исключительно через вызовы функций отдельно оформленных библиотечных модулей.
- Необходимо предусмотреть реализацию, как минимум двух модулей: один на C++, второй — Python.
- Для получения дополнительных баллов, необходима реализация двух вариантов C++ модуля: с использованием структур на динамической памяти и с использованием контейнеров и адаптеров STL.
- Интерфейс должен адекватно реагировать на попытку выполнить некорректную операцию (например, Pop из пустого стека), выводя понятное сообщение, без аварийного завершения программы.
- Визуализации должна обновляться после каждой успешной операции.

Исходный код файла приложения main.py

```
import tkinter as tk
from tkinter import ttk
from tkinter.messagebox import showinfo, showerror
import ctypes
import os
from deque_python import DequePython

cpp_ok = False
stl_ok = False
cpp_lib = None
cur_path = os.path.dirname(__file__)
cpp_lib = ctypes.CDLL(os.path.join(cur_path, "deque_cpp.dll"))
cpp_lib.deque_create.argtypes = []
cpp_lib.deque_create.restype = ctypes.c_void_p
cpp_lib.deque_destroy.argtypes = [ctypes.c_void_p]
cpp_lib.deque_destroy.restype = None
cpp_lib.deque_push_front.argtypes = [ctypes.c_void_p, ctypes.c_int, ctypes.c_char_p]
cpp_lib.deque_push_front.restype = None
cpp_lib.deque_push_back.argtypes = [ctypes.c_void_p, ctypes.c_int, ctypes.c_char_p]
cpp_lib.deque_push_back.restype = None
cpp_lib.deque_pop_front.argtypes = [ctypes.c_void_p]
cpp_lib.deque_pop_front.restype = ctypes.c_int
cpp_lib.deque_pop_back.argtypes = [ctypes.c_void_p]
cpp_lib.deque_pop_back.restype = ctypes.c_int
cpp_lib.deque_get_size.argtypes = [ctypes.c_void_p]
cpp_lib.deque_get_size.restype = ctypes.c_int
cpp_lib.deque_is_empty.argtypes = [ctypes.c_void_p]
cpp_lib.deque_is_empty.restype = ctypes.c_int
cpp_lib.deque_to_string.argtypes = [ctypes.c_void_p]
cpp_lib.deque_to_string.restype = ctypes.POINTER(ctypes.c_char)
cpp_lib.deque_free_string.argtypes = [ctypes.POINTER(ctypes.c_char)]
cpp_lib.deque_free_string.restype = None
cpp_lib.deque_clear.argtypes = [ctypes.c_void_p]
```

```
cpp_lib.deque_clear.restype = None
cpp_ok = True
import deque_stl
stl_ok = True
cur_mod = "Python"
dq = None
dq_handle = None
cur_lib = None
stl_dq = None

def make_deque():
    global dq, dq_handle, cur_lib, stl_dq
    if cur_mod == "Python":
        dq = DequePython()
        dq_handle = None
        cur_lib = None
        stl_dq = None
    elif cur_mod == "C++":
        if cpp_ok:
            dq = None
            stl_dq = None
            cur_lib = cpp_lib
            dq_handle = cpp_lib.deque_create()
        else:
            showerror("Ошибка", "Модуль C++ не найден!")
            return False
    elif cur_mod == "STL":
        if stl_ok:
            dq = None
            dq_handle = None
            cur_lib = None
            stl_dq = deque_stl.DequeSTL()
        else:
            showerror("Ошибка", "Модуль STL не найден!")
            return False
    return True
```

```
def do_push_front(num1, txt):
    if cur_mod == "Python":
        dq.push_front(num1, txt)
    elif cur_mod == "C++":
        cur_lib.deque_push_front(dq_handle, num1, txt.encode('utf-8'))
    elif cur_mod == "STL":
        stl_dq.push_front(num1, txt)

def do_push_back(num1, txt):
    if cur_mod == "Python":
        dq.push_back(num1, txt)
    elif cur_mod == "C++":
        cur_lib.deque_push_back(dq_handle, num1, txt.encode('utf-8'))
    elif cur_mod == "STL":
        stl_dq.push_back(num1, txt)

def do_pop_front():
    if cur_mod == "Python":
        return dq.pop_front()
    elif cur_mod == "C++":
        return cur_lib.deque_pop_front(dq_handle) == 1
    elif cur_mod == "STL":
        return stl_dq.pop_front()

def do_pop_back():
    if cur_mod == "Python":
        return dq.pop_back()
    elif cur_mod == "C++":
        return cur_lib.deque_pop_back(dq_handle) == 1
    elif cur_mod == "STL":
        return stl_dq.pop_back()

def do_get_size():
    if cur_mod == "Python":
        return dq.get_size()
```

```
    elif cur_mod == "C++":
        return cur_lib.deque_get_size(dq_handle)
    elif cur_mod == "STL":
        return stl_dq.get_size()

def do_is_empty():
    if cur_mod == "Python":
        return dq.is_empty()
    elif cur_mod == "C++":
        return cur_lib.deque_is_empty(dq_handle) == 1
    elif cur_mod == "STL":
        return stl_dq.is_empty()

def do_to_string():
    if cur_mod == "Python":
        return dq.to_string()
    elif cur_mod == "C++":
        res = cur_lib.deque_to_string(dq_handle)
        if res:
            s = ctypes.cast(res, ctypes.c_char_p).value.decode('utf-8')
            cur_lib.deque_free_string(res)
            return s
        return ""
    elif cur_mod == "STL":
        return stl_dq.to_string()

def do_clear():
    if cur_mod == "Python":
        dq.clear()
    elif cur_mod == "C++":
        cur_lib.deque_clear(dq_handle)
    elif cur_mod == "STL":
        stl_dq.clear()

def pick_python():
    global cur_mod
```

```
    cur_mod = "Python"
    lbl_mod['text'] = "Текущий модуль: Python"
    if make_deque():
        refresh()
        showinfo("Инфо", "Подключен модуль Python")

def pick_cpp():
    global cur_mod
    if not cpp_ok:
        showerror("Ошибка", "Модуль C++ не найден!")
        return
    cur_mod = "C++"
    lbl_mod['text'] = "Текущий модуль: C++"
    if make_deque():
        refresh()
        showinfo("Инфо", "Подключен модуль C++")

def pick_stl():
    global cur_mod
    if not stl_ok:
        showerror("Ошибка", "Модуль STL не найден!")
        return
    cur_mod = "STL"
    lbl_mod['text'] = "Текущий модуль: STL"
    if make_deque():
        refresh()
        showinfo("Инфо", "Подключен модуль STL (pybind11)")

def btn_push_back():
    if dq is None and dq_handle is None and stl_dq is None:
        showerror("Ошибка", "Сначала выберите модуль!")
        return
    num1 = ent_num.get()
    txt = ent_txt.get()
    if num1 == "" or txt == "":
        showinfo("Внимание", "Заполните все поля!")
```

```
        return
    num2 = int(num1)
    if len(txt) > 9:
        showinfo("Внимание", "Текст макс 9 символов!")
        return
    do_push_back(num2, txt)
    clear_fields()
    refresh()

def btn_push_front():
    if dq is None and dq_handle is None and stl_dq is None:
        showerror("Ошибка", "Сначала выберите модуль!")
        return
    num1 = ent_num.get()
    txt = ent_txt.get()
    if num1 == "" or txt == "":
        showinfo("Внимание", "Заполните все поля!")
        return
    num2 = int(num1)
    if len(txt) > 9:
        showinfo("Внимание", "Текст макс 9 символов!")
        return
    do_push_front(num2, txt)
    clear_fields()
    refresh()

def btn_pop_back():
    if dq is None and dq_handle is None and stl_dq is None:
        showerror("Ошибка", "Сначала выберите модуль!")
        return
    if do_is_empty():
        showinfo("Внимание", "Дек пуст!")
        return
    do_pop_back()
    refresh()
```

```
def btn_pop_front():
    if dq is None and dq_handle is None and stl_dq is None:
        showerror("Ошибка", "Сначала выберите модуль!")
        return
    if do_is_empty():
        showinfo("Внимание", "Дек пуст!")
        return
    do_pop_front()
    refresh()

def clear_fields():
    ent_num.delete(0, tk.END)
    ent_txt.delete(0, tk.END)

def btn_clear():
    if dq is None and dq_handle is None and stl_dq is None:
        showerror("Ошибка", "Сначала выберите модуль!")
        return
    do_clear()
    clear_fields()
    refresh()
    showinfo("Инфо", "Дек очищен")

def refresh():
    txt_out.delete(1.0, tk.END)
    txt_out.insert(tk.END, "Модуль: " + cur_mod + "\n")
    txt_out.insert(tk.END, "=" * 50 + "\n")
    txt_out.insert(tk.END, "Содержимое дека:\n")
    txt_out.insert(tk.END, "-" * 50 + "\n")
    if dq is not None or dq_handle is not None or stl_dq is not None:
        s = do_to_string()
        if s:
            txt_out.insert(tk.END, s)
        else:
            txt_out.insert(tk.END, "(пусто)\n")
        txt_out.insert(tk.END, "-" * 50 + "\n")
```

```
        txt_out.insert(tk.END, "Размер: " + str(do_get_size()) + " элементов")
    else:
        txt_out.insert(tk.END, "Модуль не выбран")

def about():
    msg = "Демонстрация работы с деком\n\n"
    msg += "Модули:\n"
    msg += "  Python: Да\n"
    if cpp_ok:
        msg += "  C++ (ctypes): Да\n"
    else:
        msg += "  C++ (ctypes): Нет\n"
    if stl_ok:
        msg += "  STL (pybind11): Да\n"
    else:
        msg += "  STL (pybind11): Нет\n"
    showinfo("О программе", msg)

win = tk.Tk()
win.title("Работа с деком")
win.geometry("600x500")
lib_menu = tk.Menu()
lib_menu.add_command(label="Python", command=pick_python)
lib_menu.add_command(label="C++", command=pick_cpp)
lib_menu.add_command(label="STL", command=pick_stl)
hlp_menu = tk.Menu()
hlp_menu.add_command(label="О программе", command=about)
m_menu = tk.Menu()
m_menu.add_cascade(label="Библиотеки", menu=lib_menu)
m_menu.add_cascade(label="Справка", menu=hlp_menu)
win.config(menu=m_menu)
lbl_mod = tk.Label(win, text="Модуль не выбран", font=("Arial", 12, "bold"))
lbl_mod.pack(pady=10)
frm_in = tk.LabelFrame(win, text="Ввод данных", font=("Arial", 10, "bold"))
frm_in.pack(pady=10, padx=20, fill="x")
frm_num = tk.Frame(frm_in)
```

```
frm_num.pack(pady=5, fill="x")
lbl_num = tk.Label(frm_num, text="Целое число:", width=25, anchor="w")
lbl_num.pack(side=tk.LEFT, padx=5)
ent_num = tk.Entry(frm_num, width=20)
ent_num.pack(side=tk.LEFT, padx=5)
frm_txt = tk.Frame(frm_in)
frm_txt.pack(pady=5, fill="x")
lbl_txt = tk.Label(frm_txt, text="Строка (макс 9 символов):", width=25, anchor="w")
lbl_txt.pack(side=tk.LEFT, padx=5)
ent_txt = tk.Entry(frm_txt, width=20)
ent_txt.pack(side=tk.LEFT, padx=5)
frm_btn = tk.LabelFrame(win, text="Операции", font=("Arial", 10, "bold"))
frm_btn.pack(pady=10, padx=20, fill="x")
b1 = tk.Button(frm_btn, text="Push_back", command=btn_push_back, width=12)
b1.grid(row=0, column=0, padx=10, pady=10)
b2 = tk.Button(frm_btn, text="Push_front", command=btn_push_front, width=12)
b2.grid(row=0, column=1, padx=10, pady=10)
b3 = tk.Button(frm_btn, text="Pop_back", command=btn_pop_back, width=12)
b3.grid(row=0, column=2, padx=10, pady=10)
b4 = tk.Button(frm_btn, text="Pop_front", command=btn_pop_front, width=12)
b4.grid(row=0, column=3, padx=10, pady=10)
b5 = tk.Button(frm_btn, text="Очистить", command=btn_clear, width=54)
b5.grid(row=1, column=0, columnspan=4, pady=10)
lbl_out = tk.Label(win, text="Результат:", font=("Arial", 10, "bold"))
lbl_out.pack(pady=(10, 5))
txt_out = tk.Text(win, height=12, width=70)
txt_out.pack(pady=5, padx=20, fill="both", expand=True)
refresh()
win.mainloop()
```

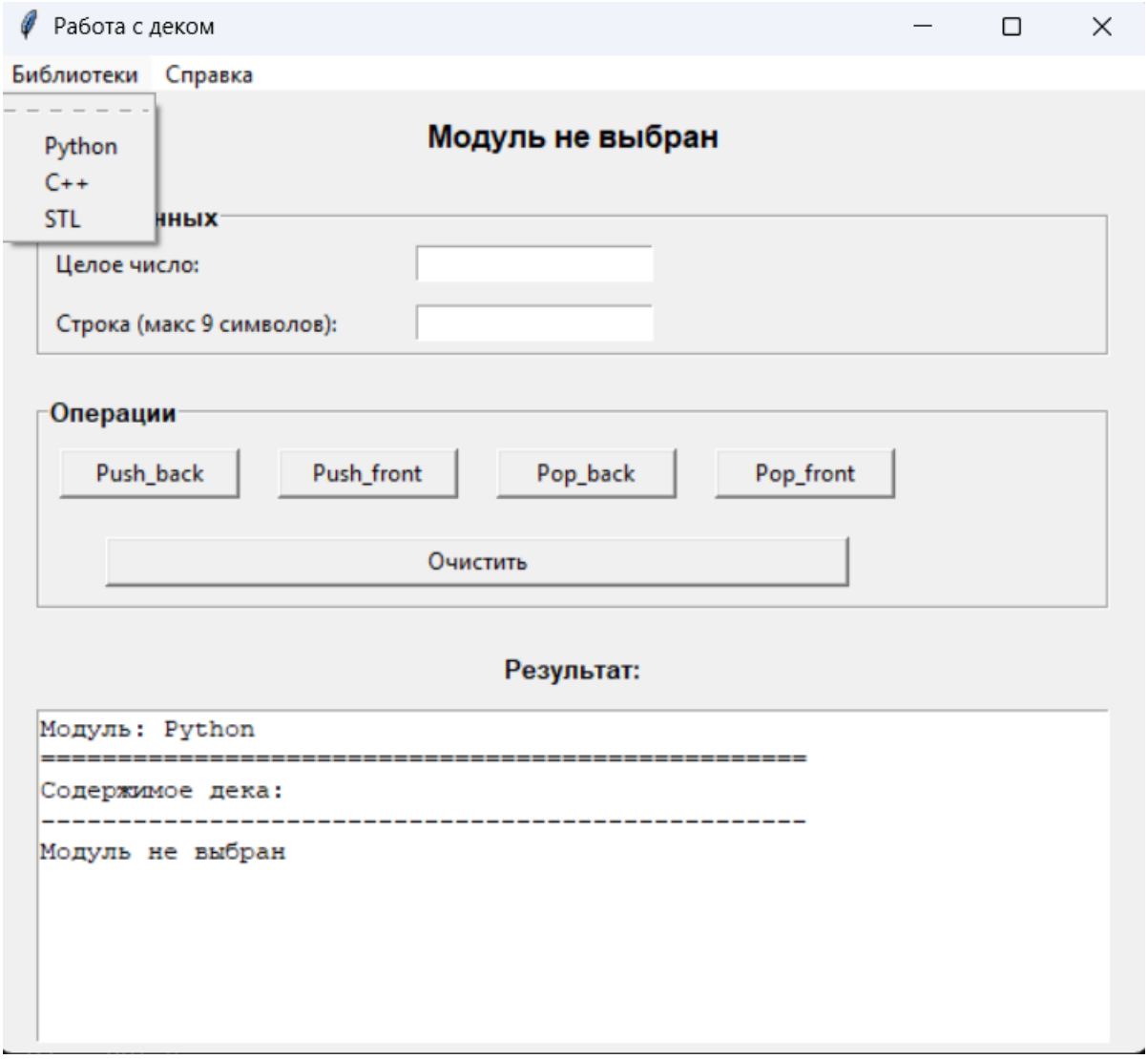

Рис. 1: Интерфейс

Исходный код модуля реализации дека deque_cpp.cpp

```
#include "deque_cpp.h"
#include <cstdlib>
#include <cstring>
#include <cstdio>

EXPORT Deque* deque_create() {
    Deque* d = (Deque*)malloc(sizeof(Deque));
    if (d == NULL) return NULL;
    memset(d, 0, sizeof(Deque));
```

```
    d->head = NULL;
    d->tail = NULL;
    d->size = 0;
    return d;
}

EXPORT void deque_destroy(Deque* d) {
    if (d == NULL) return;
    Node* cur = d->head;
    while (cur != NULL) {
        Node* temp = cur;
        cur = cur->next;
        free(temp);
    }
    free(d);
}

EXPORT void deque_push_front(Deque* d, int num, const char* text) {
    if (d == NULL) return;
    Node* n = (Node*)malloc(sizeof(Node));
    if (n == NULL) return;
    memset(n, 0, sizeof(Node));
    n->num = num;
    strncpy(n->text, text, 9);
    n->text[9] = '\0';
    n->next = d->head;
    d->head = n;
    if (d->tail == NULL) {
        d->tail = n;
    }
    d->size++;
}

EXPORT void deque_push_back(Deque* d, int num, const char* text) {
    if (d == NULL) return;
    Node* n = (Node*)malloc(sizeof(Node));
```

```
    if (n == NULL) return;
    memset(n, 0, sizeof(Node));
    n->num = num;
    strncpy(n->text, text, 9);
    n->text[9] = '\0';
    n->next = NULL;
    if (d->tail != NULL) {
        d->tail->next = n;
        d->tail = n;
    }
    else {
        d->head = d->tail = n;
    }
    d->size++;
}

EXPORT int deque_pop_front(Deque* d) {
    if (d == NULL || d->head == NULL) return 0;
    Node* temp = d->head;
    d->head = d->head->next;
    free(temp);
    if (d->head == NULL) {
        d->tail = NULL;
    }
    d->size--;
    return 1;
}

EXPORT int deque_pop_back(Deque* d) {
    if (d == NULL || d->head == NULL) return 0;
    if (d->head == d->tail) {
        free(d->head);
        d->head = d->tail = NULL;
        d->size--;
        return 1;
    }
```

```
    Node* cur = d->head;
    while (cur->next != d->tail) {
        cur = cur->next;
    }
    free(d->tail);
    d->tail = cur;
    d->tail->next = NULL;
    d->size--;
    return 1;
}

EXPORT int deque_get_size(Deque* d) {
    if (d == NULL) return 0;
    return d->size;
}

EXPORT int deque_is_empty(Deque* d) {
    if (d == NULL) return 1;
    return d->head == NULL ? 1 : 0;
}

EXPORT char* deque_to_string(Deque* d) {
    if (d == NULL || d->size == 0) {
        char* empty = (char*)malloc(1);
        if (empty) empty[0] = '\0';
        return empty;
    }
    char* buf = (char*)malloc(d->size * 100 + 1);
    if (buf == NULL) return NULL;
    buf[0] = '\0';
    char line[100];
    Node* cur = d->head;
    int index = 0;
    while (cur != NULL) {
        sprintf(line, "[%d] (%d, '%s')\n", index, cur->num, cur->text);
        strcat(buf, line);
```

```
        cur = cur->next;
        index++;
    }
    return buf;
}

EXPORT void deque_free_string(char* str) {
    if (str != NULL) {
        free(str);
    }
}

EXPORT void deque_clear(Deque* d) {
    if (d == NULL) return;
    while (d->head != NULL) {
        deque_pop_front(d);
    }
}
```

Исходный код заголовочного файла deque_cpp.cpp (deque_cpp.h)

```
#ifndef DEQUE_CPP_H
#define DEQUE_CPP_H

#ifdef _WIN32
#define EXPORT __declspec(dllexport)
#else
#define EXPORT
#endif

typedef struct Node {
    int num;
    char text[10];
    struct Node* next;
} Node;
```

```
typedef struct Deque {
    Node* head;
    Node* tail;
    int size;
} Deque;

#ifdef __cplusplus
extern "C" {
#endif
    EXPORT Deque* deque_create();
    EXPORT void deque_destroy(Deque* d);
    EXPORT void deque_push_front(Deque* d, int num, const char* text);
    EXPORT void deque_push_back(Deque* d, int num, const char* text);
    EXPORT int deque_pop_front(Deque* d);
    EXPORT int deque_pop_back(Deque* d);
    EXPORT int deque_get_size(Deque* d);
    EXPORT int deque_is_empty(Deque* d);
    EXPORT char* deque_to_string(Deque* d);
    EXPORT void deque_free_string(char* str);
    EXPORT void deque_clear(Deque* d);
#ifdef __cplusplus
}
#endif
#endif
```

Исходный код реализации дека на python deque_python.py

```
class Node:
    def __init__(self, number, text):
        self.number = number
        self.text = text
        self.next = None



class DequePython:
    def __init__(self):
```

```
        self.head = None
        self.tail = None
        self.size = 0

    def push_front(self, number, text):
        if len(text) > 9:
            text = text[:9]

        n = Node(number, text)
        n.next = self.head
        self.head = n

        if self.tail is None:
            self.tail = n

        self.size += 1

    def push_back(self, number, text):
        if len(text) > 9:
            text = text[:9]

        n = Node(number, text)
        n.next = None

        if self.tail:
            self.tail.next = n
            self.tail = n
        else:
            self.head = self.tail = n

        self.size += 1

    def pop_front(self):
        if self.head is None:
            return False
```

```
        temp = self.head
        self.head = self.head.next
        del temp

        if self.head is None:
            self.tail = None

        self.size -= 1
        return True

    def pop_back(self):
        if self.head is None:
            return False

        if self.head == self.tail:
            del self.head
            self.head = self.tail = None
            self.size -= 1
            return True

        current = self.head
        while current.next != self.tail:
            current = current.next

        del self.tail
        self.tail = current
        self.tail.next = None
        self.size -= 1
        return True

    def get_size(self):
        return self.size

    def is_empty(self):
        return self.head is None
```

```
    def to_string(self):
        result = ""
        current = self.head
        index = 0

        while current:
            result += f"[{index}] ({current.number}, '{current.text}')\n"
            current = current.next
            index += 1

        return result

    def clear(self):
        while self.head:
            self.pop_front()
```

Исходный код реализации дека с помощью stl deque_stl.cpp

```
#include <pybind11/pybind11.h>
#include <pybind11/stl.h>
#include <deque>
#include <string>
#include <sstream>
#include <cstring>

namespace py = pybind11;

struct Elem {
    int num;
    char txt[10];
};

class DequeSTL {
private:
    std::deque<Elem> d;
```

```
public:
    void push_front(int n, const std::string& t) {
        Elem e;
        e.num = n;
        std::strncpy(e.txt, t.c_str(), 9);
        e.txt[9] = '\0';
        d.push_front(e);
    }

    void push_back(int n, const std::string& t) {
        Elem e;
        e.num = n;
        std::strncpy(e.txt, t.c_str(), 9);
        e.txt[9] = '\0';
        d.push_back(e);
    }

    bool pop_front() {
        if (d.empty()) return false;
        d.pop_front();
        return true;
    }

    bool pop_back() {
        if (d.empty()) return false;
        d.pop_back();
        return true;
    }

    int get_size() {
        return static_cast<int>(d.size());
    }

    bool is_empty() {
        return d.empty();
    }
```

```
    std::string to_string() {
        if (d.empty()) return "";
        std::ostringstream oss;
        int i = 0;
        for (const auto& e : d) {
            oss << "[" << i << "] число: " << e.num
                << ", текст: \"" << e.txt << "\"\n";
            i++;
        }
        return oss.str();
    }

    void clear() {
        d.clear();
    }
};

PYBIND11_MODULE(deque_stl, m) {
    py::class_<DequeSTL>(m, "DequeSTL")
        .def(py::init<>())
        .def("push_front", &DequeSTL::push_front)
        .def("push_back", &DequeSTL::push_back)
        .def("pop_front", &DequeSTL::pop_front)
        .def("pop_back", &DequeSTL::pop_back)
        .def("get_size", &DequeSTL::get_size)
        .def("is_empty", &DequeSTL::is_empty)
        .def("to_string", &DequeSTL::to_string)
        .def("clear", &DequeSTL::clear);
}
```

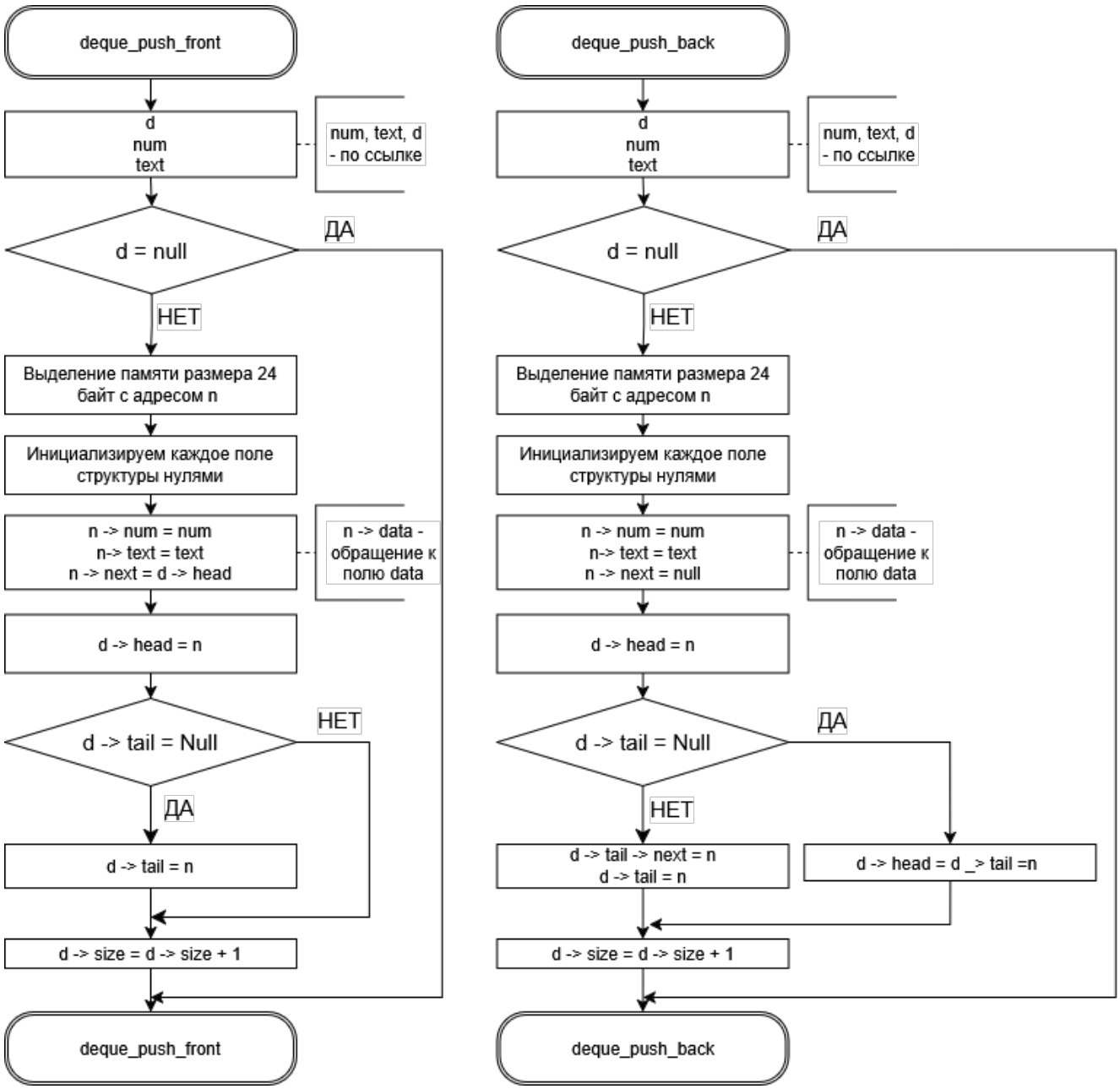

Рис. 2: Схема работы с деком. Push front/back

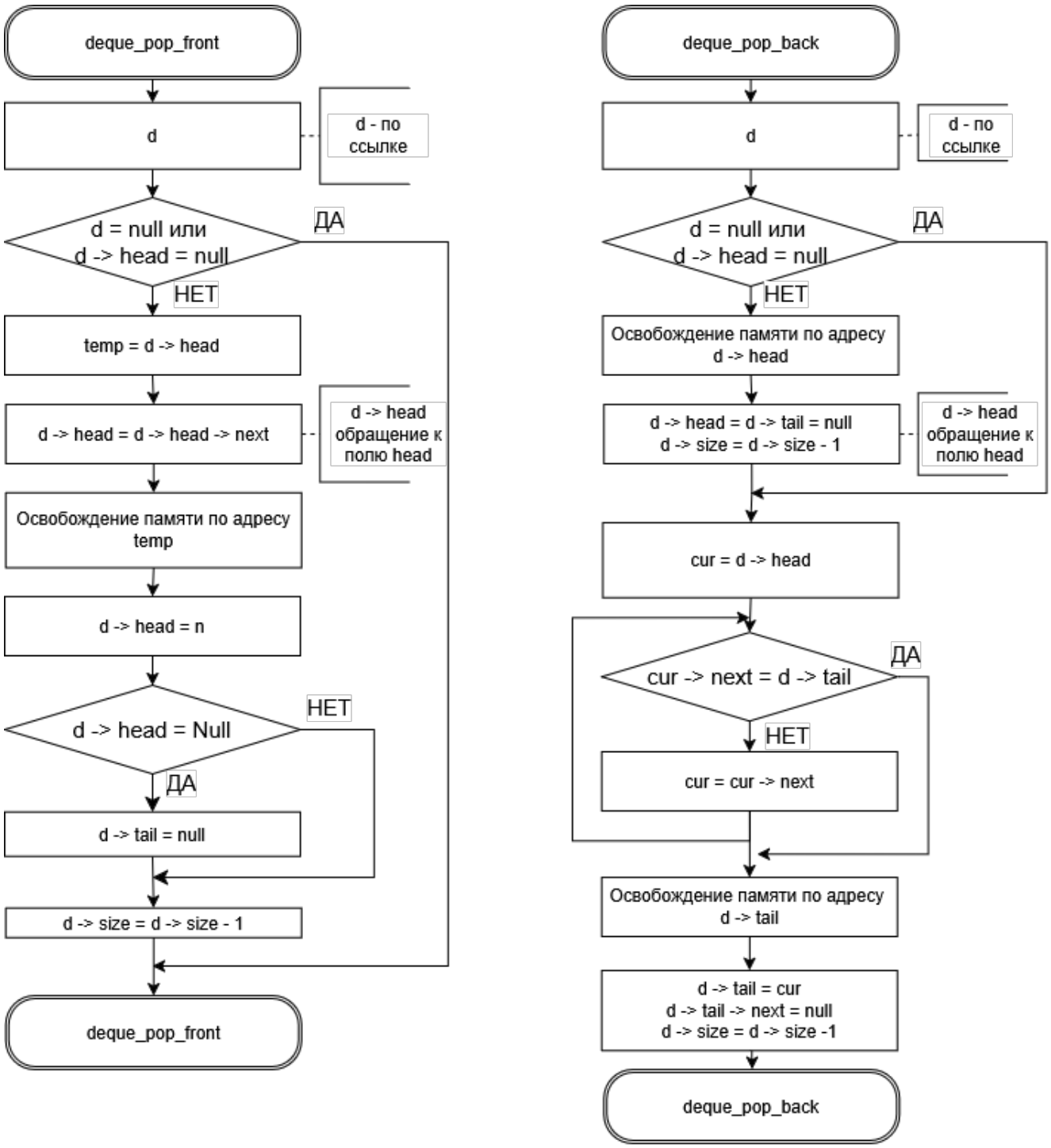

Рис. 3: Схема работы с деком. Pop front/back

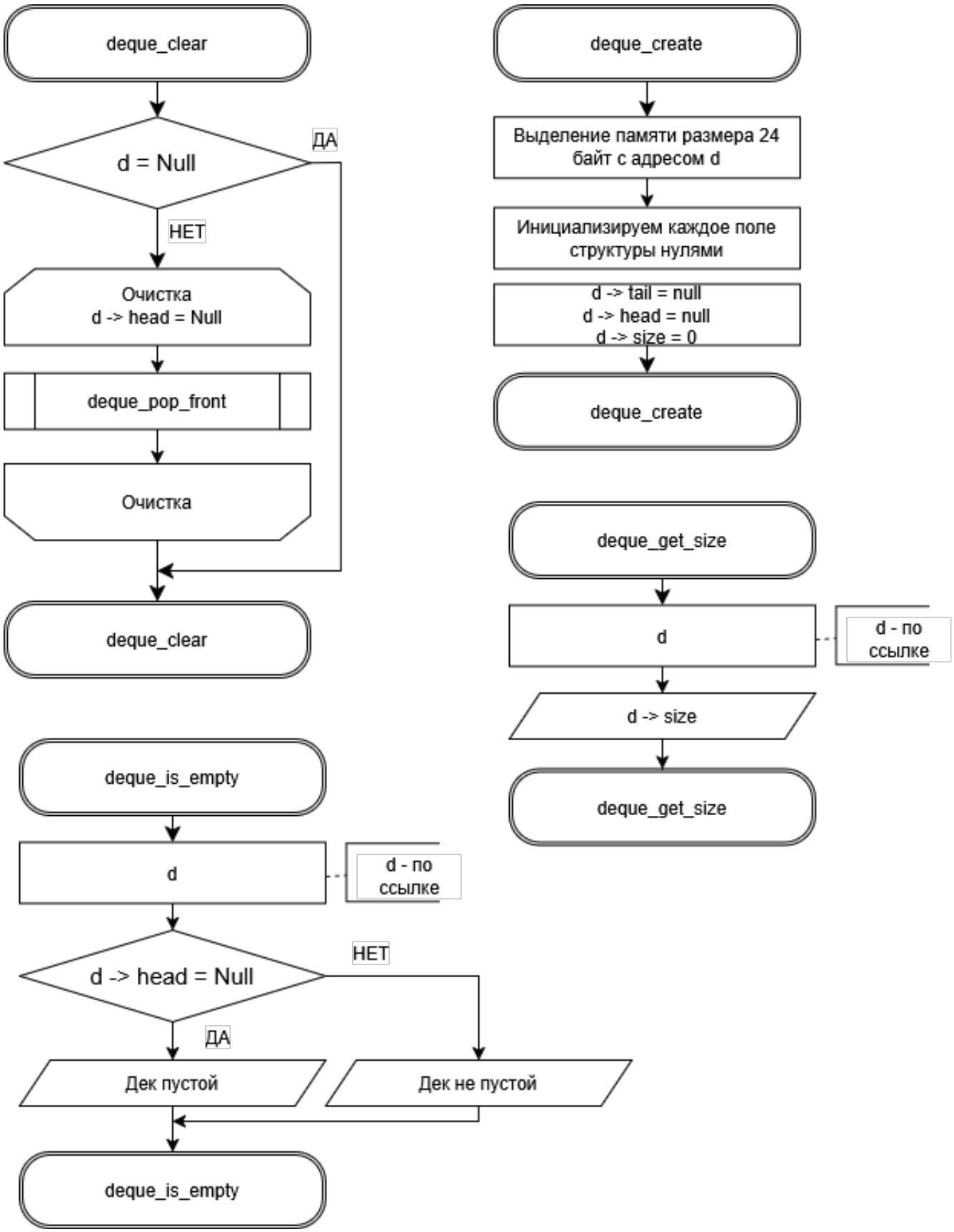

Рис. 4: Схема работы с деком. Deque create, clean, is_empty, get_size

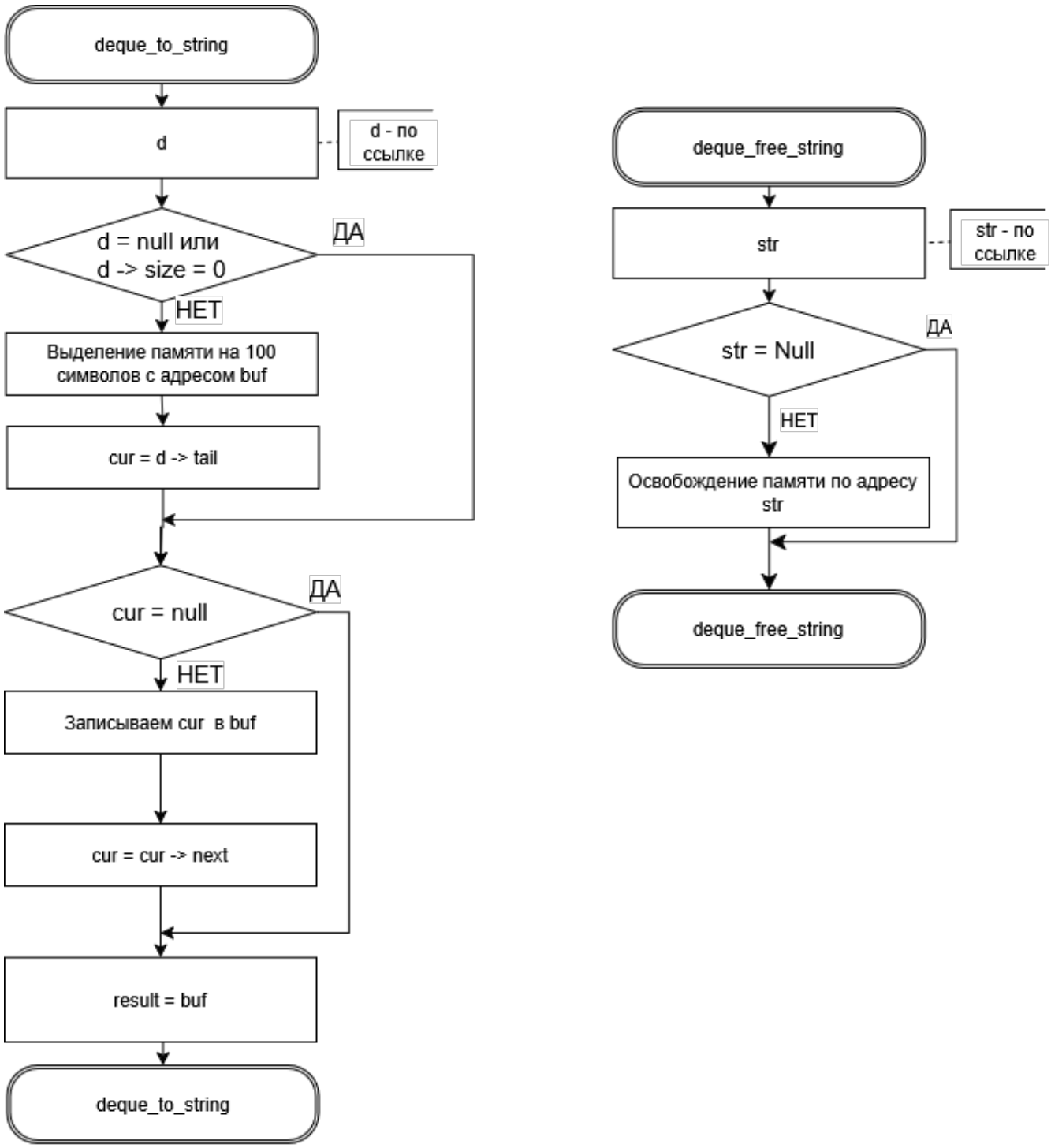

Рис. 5: Схема работы с деком. Deque to_string, free_string

Особенности работы модулей

Все три библиотеки реализуют двустороннюю очередь с одинаковым набором операций: добавление и удаление элементов с обоих концов, получение размера, проверка на пустоту, преобразование в строку и очистка. Каждый элемент хранит целое число и текстовую строку из 9 символов. В Python и C использовался односвязный список с указателями head и tail, что требует O(n) для pop_back, поскольку отсутствует ссылка на предыдущий. C++ управление памятью через malloc/free. C++ STL реализация используется std::deque контейнер из стандартного набора, где все операции с обоих концов выполняются за O(1).

Реализованные функции

Python: push_front, push_back, pop_front, pop_back, get_size, is_empty, to_string, clear
C: deque_create, deque_destroy, deque_push_front, deque_push_back, deque_pop_front, deque_pop_back, deque_get_size, deque_is_empty, deque_to_string, deque_free_string, deque_clear
C++ STL: push_front, push_back, pop_front, pop_back, get_size, is_empty, to_string, clear

Вывод

На основе разработанной структуры данных на языке C из лабораторной работы 4.1 был переписан код на C++. Сформирован модуль, который был подключен к GUI, написанного с помощью библиотеки tkinter на языке python, с помощью ctypes. Далее был написан модуль на Python и подключен к GUI. Был сформирован модуль на C++ с использванием готовых методов и контейнеров из STL и подключен с помощью PyBind11. Итого мы овладели новыми навыками, такими как: работа с PyBind, формирование статической библиотеки на C++, разработка GUI с помощью tkinter.